YARG Contributions — Roadmap

Canonical roadmap for both repos in this project (`yarg-song-server` and the `yarg` client fork). Status as of 2026-09-05.

Two tracks run in parallel. The **server track** is the #1 priority and is sequential — each phase depends on the one before it. The **client track** is independent and can absorb spare cycles at any time.

What this project is, and what it is not

Written down because the roadmap drifted once already: upstream's issue tracker is full of interesting features, and a phase named after one of them starts reading like a commitment.

What we are building:

1. **A server/client pair that talks to YARG**, so a library lives in one place and every machine in the house plays from it — instead of a full copy of the library on every device.
2. **A way to manage that library without launching YARG at all.** Browse, search, inspect, see what is broken and why, from any device on the network with the game closed and no YARG install on that device. This is a first-class goal, not a side effect of having an API.
3. **Later: chart generation** — process audio into playable charts across all difficulties, with the charts distributable separately from the audio. Phase 5, and it depends on everything above.

What we are not building: features that live inside YARG and change how the game plays. Upstream has open requests and in-flight PRs in that space — search-and-queue from a phone, selecting a song in a running client. Those are **notes for a future conversation with the YARG team**, recorded in `UPSTREAM.md`, not deliverables here. A thing being adjacent to our API is not a reason to adopt it.

Phase 0 — Foundations ✅

Repos, remotes, mirrors and the format research that everything else is built on.

- [x] `fatalexception/yarg-song-server` and `fatalexception/yarg` created on Vault2 GitLab
- [x] Song-format research spec written (`docs/research/yarg-song-formats.md`)
- [x] Architecture decision recorded (`docs/ADR-001-server-architecture.md`)
- [x] Project instructions mirrored into `CLAUDE.md` so both machines pick them up
- [x] `yarg` imported from upstream server-side: 21 branches, 172 MB of repository and 180 MB of LFS objects, default branch set to `dev`
- [x] Public push mirrors to `github.com/Coffeehedake` configured and verified: `yarg-song-server` (GitLab mirror 10) and `yarg` (mirror 11), both one-way, all branches, divergent refs not kept. First sync 2026-09-05 16:44 ET, both `finished` with no error, and the GitHub side matches GitLab commit-for-commit.

Cloned 2026-09-06, ending a deferral this document argued for twice. It is on ENG-1 at `C:\dev\YARG - Open Source Contributions\yarg` , **625 MB** on disk with submodules and LFS — the earlier "~350 MB" estimate was low by most of a factor of two, which is the ordinary fate of a size guessed rather than measured. Branch `dev` , `origin` on Vault2 GitLab, `upstream` on GitHub, `YARG.Core` submodule initialised.

The deferral's original reason had already gone void: the folder used to sit in the Syncthing `dev-projects` mesh, so a clone here replicated to r7 and Vault2 as well. Syncthing was retired on 2026-09-06, so a clone now costs disk on one machine only — and the conclusion had been surviving on the weaker of its two arguments for a day before anything acted on that.

It was cloned to answer a question rather than to start work: **YARG.Core builds and tests with plain `dotnet` and no Unity install** (`netstandard2.1` library, `net10.0` tests), which is what makes `ADR-004` groundable in real types instead of guesses.

If it needs cloning again:

```
cd "C:\dev\YARG - Open Source Contributions"
git clone https://gitlab.badassium.com/fatalexception/yarg.git yarg
cd yarg
git remote add upstream https://github.com/YARC-Official/YARG.git
git submodule update --init --recursive
git lfs pull
```

Exit criterion: both repos exist, both push to Vault2, both mirror to GitHub, and the format spec is committed. **Met 2026-09-05.**

`Coffeehedake/yarg` is a real GitHub *fork* of `YARC-Official/YARG` , not a plain repository, because that is the only shape GitHub accepts a pull request to upstream from.

Correction — LFS does propagate. An earlier revision of this roadmap claimed GitLab push mirroring drops LFS objects, and that the fork relationship was what made upstream's ~180 MB resolve. That was asserted from priors, not measured, and it is wrong.

The measurement: a throwaway GitLab project (**not** a fork, so no parent LFS storage to borrow from) with `*.png filter=lfs`, one fresh 1 MB object, push-mirrored to a throwaway **non-fork** GitHub repo. GitHub's LFS batch API returned a download action with no error, and the object downloaded from `github-cloud.githubusercontent.com` at exactly 1,048,576 bytes with a SHA-256 matching the OID. GitLab 18.11.3, git-lfs 3.7.1. Both throwaway repos were deleted afterwards.

So no special handling is needed when client work adds one of the five patterns YARG's `.gitattributes` tracks — `*.png`, `*.exr`, `*.jpg`, `*.fbx`, `*.ttf`. Which is just as well: almost any UI work in Phase 3 adds a `.png`.

The caveat that does survive: the mirror force-overwrites and the GitHub side is a fork, so anything done directly on GitHub is clobbered on the next sync. To open an upstream PR, create the branch on GitLab, let it mirror, open the PR from the mirrored branch, and leave it alone on GitHub after that.

Phase 1 — `yargsong`: the Go format library ✅

The server is worthless until Go can read and write what YARG reads and writes. This phase is pure library work with no network surface, which makes it the easiest phase to test exhaustively.

Build order (each step is independently testable):

1. **`song.ini` reader** — ✅ done (`internal/songini`). ~130 recognised keys with their types, and a deliberately lenient reader: UTF-8/UTF-16 BOMs, Latin-1 fallback so accented artist names survive, `[song]`/`[Song]`/no-header variance, trailing junk after the section bracket, values containing `=`, and malformed lines skipped rather than fatal. Two behaviours were checked against upstream rather than guessed: **duplicate keys — last wins** (upstream stores modifiers by plain dictionary assignment), and **keys and section names are lowercased** before lookup. One thing is still assumed and flagged in the source: the exact whitespace and multiple-`=` handling inside `YARGTextReader.ExtractModifierName`, which has not been read. The writer is step 5 below.
2. **`.sng` reader** — ✅ done (`internal/sng`). Implemented as an `fs.FS`, including synthesised directories, so the folder scanner and the `.sng` scanner share exactly one code path; `fstest.TestFS` enforces that. The mask-origin question the research doc flagged as unconfirmed is now settled from `SngFileStream.cs`: **the XOR index is the byte offset within each contained file, restarting at 0 per file**, not the absolute offset in the `.sng`. Upstream reaches the same result only because it decrypts whole 1 MB buffers and 1 MB is a multiple of the 256-byte table; tracking the real offset is equivalent and survives arbitrary read sizes. Malformed input is rejected with an error rather than a panic, which is tested.

Validated against the reference encoder, 2026-09-05. SngCli v0.3.0 (MIT, from `mdsitton/SngFileFormat` — the tool that defines the format) encoded a song folder we wrote, and our reader read the result: same chart bytes, same SHA-1, same metadata, and chunked streaming matching a whole-file read. The archive is committed at `internal/sng/testdata/reference-sngcli-v0.3.0.sng` so this stays a regression test rather than a thing that was true once. Until this, every `.sng` the reader had seen was produced by an encoder written from the same understanding as the reader — which proves only that the two agree with each other.

Two things the real file taught us that synthetic input could not:

- **Duplicate `song.ini` keys: last wins — confirmed independently.** A probe archive with `name = FIRST VALUE` … `name = SECOND VALUE` round-tripped through SngCli as `SECOND VALUE`, matching what we had concluded from reading `IniModifierCollection.cs`.
- **A file's extension can lie about its container.** SngCli emits audio under a `.mp3` name regardless of source format: our `song.wav` came back as `song.mp3`, byte-identical, RIFF header intact. We classify stems by name, as YARG does, so the scan is unaffected — but anything that later *decodes* audio must sniff the container, and our own writer must not reproduce this. There is a test pinning the observation.
- **Scanner** — ✅ done (`internal/scan`, `internal/catalog`). Walks a folder or a `.sng` through the same `fs.FS`, applies the chart-file priority order, classifies stems (all 14 standard, 5 clean and 4 explicit variants), resolves album art with the `cover` key overriding `album.<ext>`, finds background/video/preview, and emits the v1 catalog schema. Unrecognised files are carried in `assets.other` and every `song.ini` key is preserved in `raw_metadata`, so parse-tuning keys we do not model still survive a repack — losing those would change how a chart plays.

A test packs the same content both ways and asserts the folder and the `.sng` produce the same chart hash, metadata and parts. That is the ADR-001 "one code path" claim being enforced rather than merely intended.

Unknown intensity is `-1`, never `0`, for both an explicit `-1` and an absent key — "the charter rated this trivially easy" and "the chart did not say" must not collapse into the same value. Two `diff_*` keys (`diff_drums_real_ps`, `diff_keys_real_ps`) are deliberately left unmapped rather than guessed; the source says why.

`yarg-song-server scan <path>` walks a library and prints the catalog as JSON, so the parsers can be pointed at a real collection before anything sits behind an HTTP API. 4. **Identity** — ✅ done. `SHA1(chart file bytes)`, matching YARG's `HashWrapper`, plus a `package_hash` of our own (SHA-256 over the sorted `name:sha256` pairs) for same-chart-different-audio cases. Note the folder and `.sng` forms of one song have the same chart hash but *different* package hashes, because the folder carries `song.ini` as a file and the `.sng` carries it in the header — that is correct, and the test asserts it rather than papering over it. 5. `.sng` **writer** — ✅ done (`internal/sng/write.go`, `scan.PackDir`, `yarg-song-server pack <folder> <out.sng>`). Validated three ways, weakest first:

- **Our own round-trip** — proves only that our reader and writer agree.
- **The reference decoder.** `SngCli decode` extracted our archive with zero errors, and `notes.chart`, `album.png` and the audio all came back byte-identical to the source folder. Our archive of the same song is the same size as SngCli's, 8,806 bytes.
- **The client.** YARG v0.15.0 scanned 15 archives written by us and accepted **14**, reading every title correctly out of our metadata section — including Latin-1, UTF-16LE, a duplicate key resolving to `SECOND`, and an unknown key surviving the repack.

The one rejection was `No notes found`, and a **control settles it**: SngCli's own archive of the same source folder, scanned alongside ours in the same pass, was rejected with the identical error. The fixture's hand-written chart has no note events. As far as the client is concerned our writer is indistinguishable from the reference encoder.

Deliberate differences from the reference encoder: we do **not** rename audio to `.mp3` (SngCli does this regardless of the source container), and we refuse `song.ini` as a contained file rather than letting it disagree with the metadata section. Filenames are lowercased and collisions after lowercasing are refused, metadata keys are emitted lowercase because YARG matches `.sng` keys against a lowercase table without normalising them, and the header size is asserted against what was actually written — a mismatch there would silently corrupt every file offset. 6. **Chart preparsers** — ✅ done (`internal/chart`). Determines which parts and difficulties a chart actually contains, for both `notes.mid` and `notes.chart`. No timing, no sustains, no HOPO inference — the browse UI needs an instrument grid, and the client already has YARG.Core for everything else.

Built from documentation this time, not from source: TheNathannator's Guitar Game Chart Formats, the RBN/C3 documentation, FireFox2000000's `.chart` spec and the Elite Drums spec. The study is in `docs/research/chart-preparsing.md` with a citation per claim.

The governing rule is never to range-test a difficulty block. Every instrument family has non-playable numbers inside its own blocks, and a range test turns each into a phantom difficulty. Sixteen tests exist mostly to pin the traps the documentation names:

- force-strum and HOPO markers sit inside the block and are not notes;
- five-fret note 59 is a left-hand *animation* note unless an `ENHANCED_OPENS` event says otherwise — counting it unconditionally invents an Easy difficulty on a large share of Rock Band-derived charts;
- every Pro Keys track is *required* to carry a range-shift marker, so a test not strictly inside 48–72 reports even empty tracks as present;
- the Pro Keys animation tracks use an identical note range and differ only by name, so track names are matched whole-string, never as substrings;
- Elite Drums' modifier octave carries a disco-flip marker that exists for downcharting and can sit on a difficulty with no gems;
- a lone `HARM1` is the lead line, not a harmony arrangement.

Drum type is a heuristic because it has to be — 4-lane, Pro and 5-lane share one track — with `song.ini`'s `pro_drums` / `five_lane_drums` overriding, and both set at once recorded as the documented invalid state rather than silently resolved. Elite Drums downcharting is honoured: a song with only `PART ELITE_DRUMS` reports 4-lane, Pro and 5-lane as `derived`, because the client shows them.

The SMF reader is hand-rolled rather than a library, deliberately. Charts violate the MIDI spec in two documented ways — running status not reset after SysEx or meta events, and `0xFF` bytes inside SysEx — and a strict parser rejects files YARG plays fine.

Cross-checked against the oracle: the corpus case whose `notes.mid` YARG rejected with "No notes found" now reports zero parts from our preparser too. Same conclusion, independent route.

Exit criterion: a CLI that scans a real song library, emits a catalog, repacks to `.sng`, and YARG scans the repacked output with identical metadata and an identical hash. **Met 2026-09-05.**

`yarg-song-server scan <path>` and `yarg-song-server pack <folder> <out.sng>` do the first two. For the third: `SngCli` decoded our archives with every file byte-identical to the source, and YARG accepted 14 of the 15 archives we wrote — the one rejection being a fixture whose chart has no note events, proven by a control in which SngCli's own archive of the same folder was rejected identically.

And the parts we report now agree with the client's own verdict: on the corpus (22 cases then, 23 since archive ingest landed), **every song YARG rejects is one this scanner independently flags**, by three routes — no parts detected, `no_audio`, and `ultrastar_no_title`.

Explicitly out of scope, permanently: CON/mogg decryption, `songcache.bin` generation, `.milo_xbox` / `.png_xbox` decoding, `.yarground` inspection, full MIDI chart semantics.

Phase 2 — The server, and a sync client that needs no client changes

Two deliverables. The sync client is what makes the server *useful* before any client fork work exists, and it is the proof that the server is correct.

2a — `yarg-song-server`

- [x] **HTTP API** — `internal/httpapi`, documented in `docs/API.md`. Browse and search over the 12 attributes YARG itself sorts by, `GET /song/{chart_hash}.sng`, and a bulk `POST /api/v1/have` that takes the hashes a client holds and answers what it is missing.
- [x] **The index and the store** — `internal/library`, in memory and rebuilt at start; `internal/packcache` packs a loose folder to `.sng` on demand and caches it by package hash. Both decisions and their costs are in `docs/ADR-002-v1-store.md`.

- [x] **Sort parity with the client** — `internal/sortkey` reproduces YARG.Core's `SortString` (rich-text stripping, diacritic folding, whitespace collapse, article removal, character grouping, UTF-16 ordinal comparison) and `internal/library` orders by upstream's own comparer chains. Without this a browse list is internally consistent and unlike anything the player sees in the game.

- [x] **Ingest: loose folders, `.sng`, and `.zip` / `.7z` of a loose folder** — done 2026-09-06. `internal/scan/source.go`. `.zip` uses the standard library; `.7z` took a dependency, recorded with its measured cost in `ADR-003` (+1.62 MB, 3 → 71 compiled packages, no cloud stack despite what `go list -m all` implies).

  No adapter was needed and no scanning logic was duplicated. `archive/zip`'s Reader and sevenzip's Reader both already implement `fs.FS`, and the scanner has taken an `fs.FS` since Phase 1, so an archived song is read by exactly the code that reads a loose folder. `PackDir` became a one-line wrapper over a new `PackFS`.

  The property that matters: **a song packs to the same bytes whether it is loose, zipped or 7z'd**, so restructuring a library does not make every client re-download every song. `TestAllThreeContainersProduceTheSameArchive` asserts it directly, and the corpus now carries a zipped case (`23-zipped`) so the oracle exercises it end to end.

  RB packages are refused with a reason, not ignored — `.con`, `_rb3con`, `.pkg`, `.xex`, matched by SUFFIX because `_rb3con` files usually have no extension at all. An archive holding several songs raises `ErrTooManySongs` rather than publishing one and silently dropping the rest.

  Hardened against hostile archives, by probing rather than by reasoning — a throwaway test printed what actually happens for traversal entries, corrupt files, empty archives and odd separators, and only then were assertions written. Path traversal is dropped by `fs.ValidPath` before it can reach a served `.sng`; a corrupt archive is reported and the walk continues.

  The probe found a real defect: a zip written with **backslash** separators (`Song\song.ini`, as some Windows tools produce) surfaced a directory with nothing readable under it, resolved to `ErrNoChart`, and was **silently ignored** — a legitimate song vanishing from a library with no message, the same failure this project had already rejected for `_rb3con`. Such an archive now reports `ErrUnreadableArchive`. `internal/scan/hostile_test.go`; the shapes measured are tabulated in `TEST-CORPUS` and the reasoning is in `ADR-003`. - [x] **Config file + sane defaults** — `internal/config`. `key = value` with `#` comments, the same names as the flags, precedence defaults < file < flags actually typed, and `--write-config` to print a commented example. An unknown setting is an error, not a warning. A file NAMED on the command line and missing is fatal; the conventional `./yarg-song-server.conf` simply not existing is the normal first run and is silent. - [x] **CI** — `.gitlab-ci.yml`. `gofmt`, `go vet`, the suite, the suite again with `-race`, all six release targets, and an assertion that the Dockerfile's Go is not older than `go.mod` requires. See below for what this replaced. - [x] **A bound and an eviction policy for the pack cache** — done. `pack_cache_max`, defaulting to **2 GiB** rather than to unbounded,

with LRU eviction. The number that decided the default was measured on the vault2 deployment: 225,406 bytes of loose library produced 229,515 bytes of cache across 22 archives, **a ratio of 1.02** — so an unbounded cache over a loose library eventually needs a second copy of that library on the data disk. On the Pi target, with the library on external storage and `-data` on the SD card, that fills the card. Eviction costs a re-pack and never data, because the archive is rebuilt byte-identically and its package hash comes from the content. **That last sentence was written on 2026-09-05 and was untrue until 2026-09-06** - packing drew a random mask, so a re-pack produced a different archive; see the determinism entry below. Two smaller leaks went with it: the per-key lock map (now 256 fixed shards, bounded by construction) and orphaned `.partial` files from a crash mid-pack (now swept at start). `internal/packcache` had no tests at all; it now has six, each red-proofed. - [x] **The multi-arch container image** — `container-image` in `.gitlab-ci.yml`, pushing `linux/amd64` + `linux/arm64` to `registry.badassium.com/fatalexception/yarg-song-server`.

And it needs no emulation, which is the part worth remembering. The earlier plan here was to install qemu/binfmt on Vault2. That plan was wrong. Go cross-compiles natively, and the Dockerfile already pins its build stage with `FROM --platform=$BUILDPLATFORM`, so the toolchain runs at the host's own architecture and Go emits the arm64 binary itself. Emulation only enters if the build stage is pulled *for* the target platform, which that directive prevents. **Nothing on the Vault2 host was changed, and nothing should be.**

That correction came from the ShopStack session, who also established that `juniper-pi-deploy` — the "Pi framework" this was going to be cloned from — builds bootable SD-card images and publishes no container images at all. Same word, different problem.

Two things the job does that are not obvious:

- **It creates a `docker-container` buildx builder.** The default builder uses the `docker` driver, which cannot build more than one platform and cannot produce a manifest list at all. Without this step buildx fails with a message about the driver rather than anything that mentions multi-arch.
- **It verifies the result rather than trusting the exit code.** `ci/verify-multiarch.sh` asserts the manifest covers both platforms *and* reads the ELF machine type of the binary inside the arm64 image. An amd64 binary under an arm64 manifest entry passes every check that only reads exit codes, and then fails to start on the Pi. The check is structural rather than execution-based on purpose: running the arm64 binary in CI would need the emulation this project deliberately does not have.

That structural check was doing its job, but it is not the same as an execution, and this document said so for weeks. On 2026-09-06 the gap was closed on real hardware instead of in CI — a Raspberry Pi 4 ran both the arm64 binary and the published arm64 image. CI's job is unchanged: catch a mislabelled manifest early. Proving it *runs* stays a hardware exercise.

What CI replaced. This project had no `.gitlab-ci.yml`, so GitLab fell through to Auto DevOps. Pipeline #2216 — the only pipeline this project has ever run — failed three jobs with exit 127 trying to execute `/build/build.sh`, `/bin/herokuish` and `lsif-go`. Those are container-executor assumptions, and the runner is a **shell** executor on Vault2, which has none of them. A red pipeline nobody believes is worse than no pipeline, so the CI here brings its own pinned Go toolchain (SHA-256 verified before use, cached between runs) rather than depending on whatever happens to be installed on the host.

Measured end to end on 2026-09-05, against the 22-case corpus: the server indexed all 22 in 15 ms, `POST /have` from an empty client reported 22 missing, all 22 downloaded through `GET /song/{hash}.sng`, and re-scanning the downloaded archives gave 22 songs and 0 failures.

Then the oracle, which is the only test that means anything here: **YARG v0.15.0 scanned the 22 archives the server produced and accepted 20.** The two it refused were the two corpus cases built to be refused, and our own scanner flags both independently — `No notes found` against a preparser reporting no parts at all, and `No audio accompanying the chart file` against our `no_audio` issue. The standard holds on the served archives as well as on the folders.

One thing that run turned up, worth knowing before Phase 2b. Repacking a loose UltraStar folder whose `notes.txt` has no `#TITLE` makes YARG *accept* a song it refuses as a folder. As a folder the title has to come from the chart, and a missing `#TITLE` is fatal — "Name metadata not provided". Packed, the name lives in the `.sng` metadata section, which the packer fills from `song.ini`, so the song has a title and plays. Our scanner agrees with the client in both forms (`ultrastar_no_title` on the folder, no issue on the archive), so nothing here is broken — but it means "the server serves exactly what the folder was" is not quite true for this one format, and a sync client will show a song the player could not previously play.

2b — sync client

A small companion binary that pulls the server's library into an ordinary local songs folder. Unmodified YARG then just sees files. This ships real value in days, with zero risk to the client, and exercises the whole server end-to-end.

- [x] **`yarg-sync`** — `internal/syncclient` and `cmd/yarg-sync`, documented in `docs/SYNC-CLIENT.md`. Writes `<chart_hash>.sng` and nothing else, verifies every download by re-deriving identity from the bytes it received, resolves a shared chart hash deterministically, and leaves everything it did not write strictly alone — including under `-prune`, which is off by default. Built for all six release platforms.
- [x] **End-to-end coverage** — `internal/e2e` runs the real `httpapi.Server`, `library.Build` and `packcache` against the real client. Stubs prove one side's logic; only this catches the two sides disagreeing about the wire. All five tests were red-proofed by breaking the code they cover.
- [x] **Run it against the oracle** — done 2026-09-05. YARG v0.15.0 pointed at a folder `yarg-sync` filled: **20 of 22 accepted**, the two refusals being the two our scanner flags. "Unmodified YARG just sees files" is now a measured result rather than a claim. The run found the fifth

oracle finding and two real defects — a `notes.mid` with no note tracks was not flagged at all, and `PreparseUltraStar` skipped note derivation whenever `#TITLE` was missing, so a chart reported zero parts for a song that plays. Both fixed and red-proofed; write-up in `docs/TEST-CORPUS.md`.

- [x] **Run the server as a container, off the development machine** — done 2026-09-05 on vault2. The published image had never actually been executed before this; CI only ever checked the image config's architecture and the binary's ELF machine type, which are checks on bytes. It now runs: 8.6 MB distroless, indexed 22 songs in 7 ms, `version=827e0fe`. `yarg-sync` from ENG-1 over Tailscale pulled 22/22 in 341 ms — recorded at the time as "byte-identical to the localhost run", which was a **byte-count** comparison and not a per-file one; the files were in fact not identical, for the reason below — and a second sync transferred nothing. **Unmodified YARG on the result: 20 accepted, 2 refused** — the same two, both flagged by our scanner. Write-up in `docs/DEPLOY-VAULT2.md`.

- [x] **Execute arm64 on real ARM hardware** — done 2026-09-06 on a **Raspberry Pi 4 Model B Rev 1.5**, `aarch64`, Debian 13 trixie. Both halves ran:

  - the cross-compiled `linux/arm64` **binary**, confirmed by the Pi's own `file` as `ELF 64-bit LSB executable, ARM aarch64, statically linked`, indexing 22 songs in 24 ms;
  - the published multi-arch **container image**, `arch=arm64`, image id `3c5fdd28…`, `version=1ae2219`, indexing 22 songs in 25 ms under docker.io 26.1.5.

  Until this run, "portable to Raspberry Pi" rested entirely on ELF machine type and image config — checks on bytes, never an execution. It is now a measured result.

  And it extended the determinism result. Archives served by the arm64 server, both as a bare binary and as the container, are byte-for-byte identical to all five earlier x86-64 sync sets. Seven independent syncs, two operating systems, two CPU architectures, 154 archives, one set of bytes. The mask derivation is architecture-independent, which it had to be and which nothing had tested. Write-up in `docs/TEST-CORPUS.md`.

  Caveats stated rather than buried: the board was a **Pi 4, not the 3B+** this project had been waiting on (that board is dead — steady red PWR, no ACT activity, with three independently written cards). The Pi was a **one-shot machine, wiped afterwards**, so this is "arm64 ran on this date", not a maintained deployment. And the image reached it by `docker save` on vault2 and `docker load` on the Pi rather than a direct registry pull, to avoid putting a registry credential on a throwaway host — the image id is identical either way, so what ran is the published image; only its transport differed.

  The one-shot shape is deliberate, not a shortcut. The Pi is a *verification target*, not a development or deployment one: development happens on Docker, and a Pi is picked up periodically to confirm the arm64 build still runs on real ARM silicon. So there is no maintained Pi to keep current, and no Pi-shaped work waiting on hardware — each arm64 claim is dated, and re-measured rather than inherited. - [x] **The exit criterion, with a second client** — done 2026-09-06. YARG was installed on r7-desktop by copying ENG-1's portable install, and both

machines synced from one server and were scanned by unmodified YARG: **20 accepted, 2 refused on each, the same two songs for the same two reasons.** - [x] **Deterministic packing** — found by that run and only findable by it. The two machines received 22 archives each, 229,515 bytes each, 0 failures each — and **16 of the 22 files differed**. `sng.Write` drew its header mask from `crypto/rand` on every call, so `PackDir` was not a function of its input, and 16 was exactly the number the bounded cache had evicted and re-packed between the two syncs. That broke the strong `ETag`, broke a `Range` resume spanning an eviction, and made "two machines syncing one server get the same bytes" false wherever this repo asserted it. The mask is now derived from the package hash (`sng.MaskKeyFor`); it is stored in plaintext in the header regardless, so deriving it gives nothing away. **22/22 identical across a full cache wipe and across the two machines.** Two of the things that hid it are worth more than the fix: a same-song-twice test that was always a cache *hit* and therefore passed whatever the packer did, and `TestWriteUsesAFreshMask`, which actively *required* the non-determinism. Write-up in `docs/TEST-CORPUS.md` . - [x] **Re-measure the containerised chain against the fix** — done 2026-09-06. vault2 re-pulled onto `c623dae` with its pack cache wiped, then five independent syncs compared file by file: ENG-1 and r7 against the working-tree server, ENG-1 against that server after a full cache wipe, and both machines against the container. **110 archives, all one set of bytes.** Two servers built for different operating systems by different toolchains agree, so **the mask derivation is not platform-dependent** — which it had to be, and which nothing had shown. Unmodified YARG on the container's output: 20 accepted, 2 refused, the same two. - [x] **Deploy the hardening and measure it on the real server** — done 2026-09-06. vault2 re-pulled onto `077f36e` , pack cache wiped, all 23 songs re-packed by the new binary: **byte-for-byte identical to the `423902b` cache**, and the 23 files a `yarg-sync` then received hashed to the same set as the 23 packs on the server. Every earlier determinism result compared machines or architectures at a *fixed* commit; this one holds **across a code change**, which is the case an `ETag` and a `Range` resume actually meet, since a server is upgraded far more often than it changes CPU.

And the defect was reproduced on the deployment before being declared fixed. A probe archive with backslash separators was dropped into the host library: the server indexed 23 songs with `problems=1` , naming the file and saying how to fix it, in the log and in `/api/v1/library` both. The previous image reported `problems=0` and said nothing. Probe removed, library restored to 23 clean cases. See `docs/DEPLOY-VAULT2.md` . - [x] **Measure what this costs at real library sizes** — done 2026-09-06, `cmd/mkscale` and `docs/SCALE.md` . Every number in this repository until now came from 23 songs and 238 KB. Now: **index is linear**, 0.594 / 0.583 / 0.569 ms per song at 1,000 / 10,000 / 31,109 songs, and memory fits **13 MB + 6.5 KB per song** across the whole range.

Those constants extrapolate, and one of them is a constraint on goal #1: **a 100,000-song library needs ~665 MB resident**, so a 1 GB Pi 3B+ could not hold it while serving from it and a 4 GB Pi could. The catalog is in memory by design (ADR-002); this is the number that says where that design runs out.

On the byte axis — 200 songs of 5 MB audio, 1.07 GB — **resident memory stayed at 16.5 MB while serving 1.17 GB**, so packs stream rather than being assembled in memory, which the code was written to do and nothing had measured. With the cache bound set to 100 MB against that 1.17 GB library, **all 200 songs were still delivered and the cache never exceeded 102 MB** — the `pack_cache_max` property re-measured under 11x pressure rather than trusted.

The library is generated, not downloaded: community charts carry copyrighted audio, and a corpus that cannot be rebuilt on another machine is not a corpus. **What this does not close:** the songs are uniform, so a real library's *variety* is untested, and the oracle has never run at scale — that is the measurement most likely to find a YARG rejection category the scanner misses, and it needs real charts. - [x] **The oracle, against REAL charts and as a script** — done 2026-09-07. `scripts/oracle.ps1` replaces a documented hand procedure: it repoints YARG's `SongFolders`, wipes the cache, launches, waits, compares both verdicts, and restores `settings.json` in a `finally` so a crash still puts the operator's YARG back. It waits on the **song cache** rather than `badsongs.txt`, because a library YARG is happy with writes no `badsongs.txt` at all, and it requires that file to be newer than the launch so last run's verdict can never be read as this one's.

Run against **128 real community songs** (1.16 GB, supplied by Jay): YARG refused 0, we flagged 0. **A weak positive, and recorded as one** — both sides agreeing means there were no disagreements available to find, and a curated pack people actually play is close to the least informative sample for this test. What it does establish is **zero false positives on real charts**, which nothing had tested, and that `ErrTooManySongs` is right about real pack `.zip`s rather than only synthetic ones. - [x] **Un-skip upstream's own scanner tests** — done 2026-09-07. `YARG.Core.UnitTests`' `FullScan` and `QuickScan` skip unless `YARG_TEST_SONG_DIRS` names a song directory. Pointed at the same 128 songs, both pass and YARG.Core writes no `badsongs.txt` — a second oracle that runs **in-process in seconds**, needing only the .NET SDK rather than the game. Upstream's own comment says the only fail condition is an unhandled exception, so it is a crash test over real input, not a verdict test; `docs/TEST-CORPUS.md` says so rather than letting it read as more.

Still uncovered: real-world `.chart` coverage is zero — and `.chart` is the format whose early-return bug this project already found once.

Re-measured 2026-09-07 and it is worse than "all 128 songs are `.mid`". The three multi-song `.zip` packs the scanner refuses were opened and counted too: **270 songs in the corpus, 128 of which are `.chart`-free loose folders and 128 inside the packs, and every single chart file is `notes.mid`. Zero `.chart`, zero `notes.txt`.**

Onyx is not the route, and the handoff that said it was should be ignored. Its release notes describe `.chart` as an *import* format and its conversions as producing CH-compatible output (`song.ini` + `notes.mid`) — the wrong direction. A converted chart would also be the converter's dialect rather than what charters actually publish, which is most of the point of wanting real ones.

What would actually close it: a handful of genuine community `.chart` songs dropped into `C:\dev\_incoming\YARG` — Clone Hero-era charts are commonly `.chart`. That is a two-minute job for a human and one this session cannot do for itself. - [x] **Break real songs on purpose, and find something** — done 2026-09-07. `cmd/mkbroken` damages songs with real structure in 16 named ways, each carrying a **predicted verdict**, so the oracle compares three columns rather than two: prediction, our scanner, YARG.

The standard failed, for the first time on this axis. Of 6 songs YARG refused, **2 were ones we passed with no issue at all**: a MIDI truncated to a third of its length, and a plain text file named `notes.mid`. Both draw "*Corruption of either the ini file or chart/mid file*" from YARG. The cause was written down that day as "*the scanner never parses the chart, it only hashes it*" — **and that diagnosis was wrong**; see the next entry, which is the more useful half of this one. - [x] **Decide what "is this even a chart?" should mean** — done 2026-09-07, and the answer was not the one that had been designed.

The proposal was a structural check: `MThd` magic on `notes.mid`, a `[Song]` section in a `.chart`. Probing eleven damaged shapes first — printing every issue rather than reasoning about them — showed that **half of it already existed and the other half would have missed the case that mattered.**

The scanner has preparsed every chart since Phase 1. For an unreadable chart it *detected* the failure and recorded a cosmetic `parts_note`, which is indistinguishable from not detecting it; that is now the issue `chart_unreadable`. For a `.chart` that is not a chart, `no_notes` already fired, so that half of the proposal was unnecessary. And the `MThd` check would have passed `03-chart-truncated` cleanly, exactly as the warning above said: a real nine-track chart cut to a third keeps a valid header and several **complete** `MTrk` chunks, so it preparses fine and reports genuine instruments.

The check that does work is the file contradicting **its own chunk lengths** — issue `chart_truncated`. It does not catch a cut landing exactly on a chunk boundary, which is byte-for-byte a valid chart with fewer tracks; that limit is pinned as a test rather than left as a comment.

Measured: 270 real songs, 2 flagged — precisely the two damaged on purpose. Oracle run 8: 6 refused, **0 missed**, standard held. - [x] **Measure what several clients at once do — and fix what it found** — done 2026-09-07. Every measurement before this was serial, while the whole point of the project is a server on a LAN with more than one client. The probe found a **real defect**: the song handler asked the pack cache for a *path* and opened it as a second step, so an evicting goroutine could remove the archive in between and the server answered **404 "this song is no longer where the index says it is; rescan"** for a song that was present. Measured at 64 clients over a 40-song library bounded to two archives: **2, 1 and 0 spurious 404s across three runs of 2,560 requests** — rare enough never to be met by hand, constant on a busy server, and it points the operator at a library that is fine.

`packcache.Open` now returns an **open file**, so there is never a moment where the caller holds a name but not a handle. Two properties held under the same pressure: the thundering herd collapses (64 concurrent requests for one cold song → one pack, one archive, 64 identical responses), and **eviction costs a re-pack and never data** — 7,680 requests against a cache bounded to a *single* archive returned byte-identical archives, zero mismatches. One promise was corrected: `pack_cache_max` is enforced after each insert, so it is a **high-water target rather than a hard cap**, overshooting by one to three archives as concurrency rises. Details in `ADR-002`; the instrument mistakes are in `SCALE.md`.

Resolved, and worth remembering for the response rather than the event: on 2026-09-05 a Defender machine-learning verdict (`Trojan:Win32/Bearfoos.A!ml`) quarantined `cmd/yarg-sync` builds for about a minute, then stopped reproducing within the hour with nothing changed on the machine — verified across three forced relinks and three distinct binary hashes, zero new detections, same signature version, no exclusion added. Acting on it immediately would have left a permanent Defender exclusion behind for a problem that had already evaporated. If it returns, re-measure before doing anything structural; the escalation is Microsoft's false-positive form, never an exclusion. Code signing is *not* an available escalation — it was considered and declined on 2026-09-08 (free software, no certificate subscription). Details in `docs/SYNC-CLIENT.md`.

Exit criterion: a Pi on the LAN serves a shared library; two machines running stock YARG both see the same songs without either one being modified.

Phase 3 — Native remote song source in the client fork

Only now does the `yarg/` fork get touched. This is the upstream-facing work.

- Add an HTTP song source alongside local folders — browse, stream, cache.
- Touches YARG's song cache, so it must be designed with upstream in mind rather than bolted on.
- PRs target `dev`. Never `master`.

Decided 2026-09-06: build it in the fork now, and approach upstream in parallel. Their answer shapes this work; it does not block it. That is a deliberate choice rather than impatience — their own contributing guide invites experimentation on undecided features and says large progress can promote a tier, so a working implementation is a better opening than a proposal.

Reconnaissance done 2026-09-06, recorded in `UPSTREAM.md` with the draft post:

- **Their `CONTRIBUTING.md` sorts every feature into six tiers, and the tier decides whether a PR is even read.** A remote song source matches none of the published examples, so *which tier* is the first question — and asking it on Discord before building is what their guide explicitly tells contributors to do.

- **Nothing upstream proposes this.** Searched their issues: the nearest is **#860**, a built-in web server for search and queueing from a phone — a *control plane*, not a content source, and worth not conflating with this. `#1030 "user-supplied song sources"` is about source **icons**.
- **Two of our three permanent non-goals are things upstream has independently ruled out.** CON Decryption is in their Out of Scope list, verbatim: "*Do NOT PR these features. Your PR will immediately be denied.*" We are not asking them for anything they have already refused, and saying so is worth a sentence in the opening post.
- **The ask is smaller than it sounds.** The server already hands out plain `.sng` that unmodified YARG reads, so this is not "support our protocol to play our songs" — it is "fetch from a URL instead of needing a separate sync tool". The seam underneath is that `SongEntry` is abstract but `ActualLocation`, `SortBasedLocation` and `GetLastWriteTime()` all assume a local path; *a song entry whose bytes are not on disk* is a general capability rather than a feature about our server, and may be an easier thing for upstream to want.

A note, not a plan. The same client-to-server channel could carry a queue, which is the shape of upstream's #860. That is worth knowing when we eventually talk to their team, and it is not on this roadmap — see "What this project is, and what it is not" at the top. (An earlier revision of this paragraph called #860 unbuilt and said nothing could reach a running client. Both are false: #984 does exactly that, and has since February 2025.)

Design recorded 2026-09-07: `ADR-004`, grounded in YARG.Core rather than in the paragraph above. Reading the code changed the shape of the ask twice over, and the entry above was wrong in one place — it says the seam is that `ActualLocation` / `SortBasedLocation` / `GetLastWriteTime()` assume a local path. True, but not the useful finding.

- **YARG.Core contains no networking whatsoever** — `HttpClient|System.Net|UnityWebRequest` returns **zero** matches across the whole library. The Unity project does network (Discord, localization, song sources), but none of it produces a `SongEntry`. So this is not "extend the existing pattern"; there is no pattern, and putting an `HttpClient` in YARG.Core is a change of character rather than an addition.
- `SngEntry` **and** `UnpackedIniEntry` **are** `internal sealed` **with private constructors**, and so is their base `IniSubEntry`. A remote entry type cannot be added from outside the assembly at all — every entry-level approach is a change *to* YARG.Core.
- **Exactly one source seam already exists** — `protected abstract FixedArray<byte>? GetChartData(string filename)` — and it is not enough: `SngEntry` reaches for `SngFile.TryLoadFromFile(_location, …)` at seven sites and the local filesystem at nine more. `SngFile` has no stream-taking loader, though `FixedArray` does.

So the decision is **YARG.Core stays offline and the Unity layer fetches**, in three increments, with only the first committed to: a managed mirror folder inside the game (**zero YARG.Core changes** — it is `yarg-sync` moved in-process, and it needs no answer from upstream to ship); then lazy fetch-on-play, which needs exactly two named seams; then server-supplied entries, deferred

because consuming that path means speaking the cache format, whose version constant is `26_09_04_00` and whose groups store absolute paths — the `songcache.bin` non-goal wearing a hat.

The upstream ask gets much smaller as a result: not "support remote libraries" but "take `SngFile.TryLoadFromStream`, and consider a materialise hook on `IniSubEntry`" — both small, both useful on their own, neither dragging networking into the library.

Increment 1 shipped 2026-09-07 — `yarg` `6a05002` on `dev`. The game mirrors a server's library into `PathHelper.ServerLibraryPath` and appends it to the scan list beside `PathHelper.SetlistPath`; **zero YARG.Core changes**, as the ADR predicted. Verified against vault2 from batchmode, not merely compiled: 23 songs, 238,215 bytes, every chart hash checked with YARG.Core's own `SngFile` / `HashWrapper`, second run downloaded nothing. That byte count is identical to what four concurrent `yarg-sync` clients pulled from the same server, so the C# and Go clients agree exactly.

One thing the ADR did not predict, found by running it: **Unity's `insecureHttpOption` defaults to `NotAllowed` and a LAN song server is plain HTTP**, so the first smoke test failed outright. Set to `DevelopmentOnly` — enough to develop and test, nothing weaker shipped to players — and what a release build should do is now question 4 in the Discord post rather than a default quietly changed in a fork.

The same defect in `yarg-sync`, `63c4f31` — because the two clients mirror each other. Having found it in the in-game mirror, the obvious next question was whether the Go client did the same thing. It did: `postHave` returned the server's list straight to the caller and every entry became both a URL and a path. `filepath.Join` **cleans** a path but does not **confine** it.

The exposure differs by platform, and the difference is the interesting part. On POSIX the write is *transient* — `fetchOne` downloads to a `.part`, verification rejects the bytes, and `os.Remove` succeeds, so nothing is left to find. The identical sequence on Windows leaves the file, because YARG.Core holds a non-`.sng` file open and the delete fails. **Two bugs compose on one platform and not the other**, which is why the Go test asserts on the *request count* rather than on files left behind: a test whose real assertion never fires is decoration. Red-proofed by widening the guard — six song requests for names that are not chart hashes.

Go is better off than .NET in exactly one respect: `filepath.Join` does not discard its first argument when the second is rooted, so an absolute name cannot escape the way it does in C#.

`prune` was checked and is **not** affected — it iterates the local inventory, which the managed `-name` regex already filters, and only removes hashes the server does not have. A server cannot name a file for deletion.

Remote arbitrary file write in the mirror client, `yarg` `02f23f90`. The most serious defect found in this project so far, and it came from following the hostile server one question further: the client asked the server what it was missing and used the answer as **filenames**, unchecked.

```
foreach (var entry in json.Value<JArray>("missing") ?? new JArray())
    missing.Add(entry.ToString());          // no validation
...
string part = Path.Combine(destination, hash + ".sng.part");
```

`../..` escapes the mirror folder, and an **absolute** path is worse, because `Path.Combine` discards its first argument when the second is rooted — the file goes exactly where the server said. **Demonstrated with the guard temporarily widened**: a 2,048-byte attacker-controlled file written to an attacker-chosen absolute path, persisting on disk. With the guard restored: `rejected=6`, nothing outside the mirror, sync still completes.

What was shown, precisely. Only the absolute-path case was observed to land, because that directory existed; the `../../../..` names resolved to directories that do not, and `DownloadHandlerFile` does not create them. That is a limit of the demonstration, not a defence — an attacker picks a directory that exists. The first version of the test proved nothing at all, because the server 404'd the hostile names and no bytes were ever served.

The persistence needs a second bug. The download succeeds so the file is written; verification rejects it; and the cleanup delete is the one that *cannot* succeed, because YARG.Core keeps a non-`.sng` file locked. Two known bugs compose into a durable write.

"The player chose this server" is not an answer. The connection is plain HTTP on a LAN by design, so anything on the path can supply that list, and a server trusted for *content* should still not be trusted to name files on every machine that syncs from it. The server-side scanner already refuses traversal entries inside archives for exactly this reason; **the client had never been given the same treatment.** That asymmetry is the lesson worth keeping — the guard was written once, on the side where the danger was obvious.

The per-song library badge, `yarg` 2026-09-08 — and the blocker turned out to be softer than it had been written down. Every handoff since increment 1 has said the badge "needs the Unity editor open, cannot be verified headless". That is true of a badge that is a new object on the song-row prefab. It is not true of the badge itself: `GetSecondaryText` already returns rich text (`FormatAs` wraps the artist in `<color>` and `<font-weight>`), so a tag appended after it needs no prefab change at all — and rich text CAN be measured from batchmode.

So the badge rides on the artist line. That is a stated limit rather than a first draft: the prettier version is a prefab change, and a prefab change can only be looked at, never asserted. This is the version of the feature that can be checked.

The decision lives in `SongViewType.WithServerBadge`, **static and pure on purpose** — a `SongViewType` needs a `MusicLibraryMenu`, so an instance method would have been untestable headless, which is precisely how the feature came to be labelled unverifiable in the first place.

`Editor/LibraryBadgeProbe.cs` plants two corpus songs in the game's REAL mirror path (`PathHelper.ServerLibraryPath`) and two in an ordinary folder, scans both with YARG.Core's own `CacheHandler.RunScan`, and asserts the badge lands on exactly the mirrored ones. Entries built by the real scanner, not by hand: a hand-made entry would only prove the string comparison works.

Red-proofed in both directions, because a marker that appears on everything and one that appears on nothing fail differently:

Change	Result
badge every song	"*2 of 2 songs the player owns were marked as the server*'s", and the null-entry assertion caught it too
badge no song	"*2 of 2 mirrored songs are unmarked*" alone

The first version of the probe failed for a reason that had nothing to do with the badge, and it is the harness trap this project has already paid for once. It took the first four `.sng` files in the corpus; two of them vanished from the scan and the probe reported "*1 mirrored and 0 local entries*". `badsongs.txt` named the real cause: "*Corruption of either the ini file or chart/mid file*" — the message YARG produces when `song.ini` omits `song_length`, the scanner falls back to measuring the audio, and **batchmode has no working audio backend**. The probe now picks its songs by scanning the corpus first and using what survives, and exits **2 — inconclusive** rather than red if too few do.

Measured while doing it, and worth knowing before designing any other batchmode test: **only 5 of the 23 corpus songs scan in batchmode at all.** The other 18 are refused for reasons that are the harness's, not theirs.

The last unchecked server string: `package_hash`, both clients, 2026-09-08. Closing the file-write defect above left one server-supplied string still used without looking at it. When a chart hash exists in two packages the server answers **300** with the candidates, and the client puts the one it picks straight into `?package=` on the request that follows — unchecked, in C# and in Go alike.

Severity is genuinely low, and saying otherwise would be inflating it. The value reaches a URL, not a filename, and `VerifyChartHash` still rejects whatever comes back if it is not the song that was asked for. What it could do is put `&`, `#` or whitespace into a request the client makes. It is closed because it was the last one, not because it was dangerous — the interesting property is that *the same class of bug was written three times, in two languages, by trusting a server for names as well as for content.*

Both clients now check `^[0-9a-f]{16,128}$` — the same pattern `packcache` already enforces server-side — and both **skip** a malformed entry rather than failing the song, which matters more than it looks: skipping keeps the two clients choosing the *same* package, which is the entire reason the choice is deterministic. A 300 with nothing usable fails, and says so.

Red-proofed on both sides, because a check nobody has seen fail is a check of unknown meaning:

Side	Check disabled	Result
Go	`if false && !packageHash.MatchString(...)`	`TestPackageHashesThatAreNotHashesAreRefused` alone failed: *client chose [../../../../etc/passwd]*
C#	`PackageHash.IsMatch` removed from the guard	`HostileServerProbe` failed both new assertions and nothing else: the client asked the server for `../../../../yarg-probe-escape/package`, and for the all-bad listing it asked for the **empty string** — an entry that sorts below everything, which is the sort of value a check written by reasoning alone tends to miss.

The probe's hostile 300 lists `../../../../yarg-probe-escape/package` **first in sort order** (`.` is 0x2E, below `0` at 0x30), so a client that skipped the check would pick it: the assertion fails loudly rather than by luck. A second 300 lists nothing usable at all, and the client must fail that song without inventing a request — `RequestedPackages` on the fake server records what was actually asked for, rather than the probe assuming.

One existing test and the probe both had to be **corrected, not just extended**: they used toy package hashes (`aaaa`, `0001`) that the new check rightly refuses. Toy values were testing a path no real server can reach.

The sweep was broader than the guarantee, `yarg` 2026-09-08. Auditing what the mirror DELETES, rather than only what it writes, turned up a smaller mismatch in the same family. The in-game mirror never prunes and `yarg-sync`'s prune iterates the LOCAL inventory, so **a server cannot name a file for deletion in either client** — that part held, and is now stated in `SYNC-CLIENT.md` rather than left to be re-derived. But the mirror's sweep of dead partial downloads took **any** name ending `.part`, while the guarantee three lines above it says anything not named like ours belongs to the player and is never touched.

The mirror folder is one the game owns, so a stranger's `.part` in it is unlikely; the claim was still wrong, and **a guarantee that holds "almost always" is not one**. The sweep now matches `^[0-9a-f]{40}\.sng\.part$`, exactly what the download path can write, and anything else counts as the player's. The probe plants `stranger.part`, `not-a-hash.sng.part` and one ending `.sng.part.bak`, and asserts all three survive a sync against a server that keeps failing.

And the instrument was wrong before the code was, for the sixth time. Planting those files made an existing assertion fail — "*3 dead .part file(s) were there to sweep but only 1 were swept*" — which reads exactly like a broken sweep. It was `Directory.GetFiles(destination, "*.part")` counting the player's files as ours. The measurement was fixed, not the code.

The browse page was audited in the same pass and is a non-finding. It renders `song.ini` metadata from uploaded archives — content the server does not author — and `card()` escapes every field, with `encodeURIComponent` on the download link. Exactly three interpolations bypass `esc()`: a `problems.length` count, a `URLSearchParams.toString()`, and `p.intensity`, which is a Go `int8` and so cannot marshal as a string. Recorded in `API.md` as a negative so it is not re-investigated, with a note that the third is the one to re-check if `intensity` ever stops being an integer.

The download error branch was dead code, `yarg` `61cf1290`. Following the hostile probe with one more question — what does a *player* read when a download fails — turned up that they read `Unknown Error`, and then that the branch meant to say more had never run.

UnityWebRequest reports a non-200 in two ways and the difference is not obvious. A 4xx/5xx or a dropped connection makes UniTask **throw** from the await, so everything written after it — the `if (request.result != Success)` check this code used to report failures — is **unreachable**. A **300 comes back as success** with `responseCode 300`, because there is no `Location` header to follow, so the duplicate-package case must be checked after a normal return. Both are now handled on both paths so neither assumption is load-bearing. A cut-off download now says *"the connection ended after 4317 of 8634 bytes (HTTP 200)"*.

The 300 path had never met a 300. The live corpus has `duplicate_packages: 0`, so nothing had ever served this client one. The probe now does — declining to choose, listing two packages, and serving bytes only when asked for one by name — and it earned itself immediately: the first version of the fix assumed a 300 also arrived as an exception, which broke the duplicate-package path. **A regression introduced and caught inside the same change**, in a path that would otherwise have made every song existing in two packages permanently unfetchable, silently, until somebody had a duplicate.

The mirror's integrity guarantee, tested against a hostile server. `SongServerSync` claimed in its own comment that "a crash or a dropped link mid-download cannot leave a truncated archive under a name the scanner will trust" — true by inspection, never reproduced, which is precisely the standing the packcache eviction race had until CI ran the one test that could fail. `Editor/HostileServerProbe.cs` now serves the four ways a download goes wrong from a raw-socket server: a body cut in half mid-transfer (Content-Length promises the whole file), a real archive served under someone else's hash, a 500, and random bytes.

The guarantee held — no bad archive was ever named — but three real defects came out around it, none of which any unit test had reached:

1. **The cleanup delete could throw and replace the real error.** A rejected download reported "the process cannot access the file" instead of why it was rejected. The delete is now guarded and can never mask the cause.
2. **`SngFile.TryLoadFromFile` leaks its `FileStream`** when the file is not a `.sng`, so on Windows the rejected file stays locked and cannot be deleted at all. Partial downloads are now swept by `Inventory` on the next run instead, and `Result` reports `swept=`.

3. `SngFile.Dispose()` **throws on the value a failed load returns**, and that `NullReferenceException` replaced our own message — "downloaded file is not a readable .sng" reached the caller as "Object reference not set to an instance of an object".

Two of the three are upstream's, both one-liners in the same failure path, and both are now written up with reproductions in `UPSTREAM.md` as something to hand them ahead of any API request.

The probe also pins what the guarantee does **not** promise: a failed download can still leave a `.part`, because the locked file cannot be deleted. What must hold is that a `.part` is never a song and that dead ones do not accumulate — measured at 1 after one sync and 1 after two, against a server that keeps failing.

A `server:` search filter came first, `yarg` `a31ddef7`. `server:yes` shows only what the mirror brought in, `server:no` only what was already yours, and it composes with everything else (`artist:queen;server:yes`).

The signal is the path, and that was measured before anything was written. A probe scanned a real 23-song mirror and printed what a mirrored song looks like to the library. Both obvious candidates fail: `Source` is "Unknown Source" — it belongs to whoever charted the song — and `Playlist` is "Unknown Playlist" for every one, because a loose `.sng` takes its playlist from its own metadata rather than the folder it sits in. Playlist matched the mirror folder's name for **0 of 5** scanned entries; `ActualLocation` was under the mirror for **5 of 5**.

That also answered the question worth asking first: **YARG already has a `folder:` filter, and it matches on playlist** — so `folder:serverlibrary` does not do this today and the new filter is not a duplicate. Had it come back the other way, the right move would have been to write nothing and document the filter that already existed; this project has already once designed a fix whose second half turned out to be unnecessary.

Not a new `SortAttribute`, deliberately. The search pipeline keys filters on that enum, but the enum is also what the library sorts and groups by and several values already have no comparer; adding one for something that can never *be* a sort order would put a value in it that half the code must remember to ignore. `server:` is lifted out of the query before the pipeline runs and applied as one pass over the already-narrowed result.

Still open: the per-song badge. Marking mirrored songs visually in the library rows needs UI work whose result cannot be verified from a headless session. The filter is the verifiable half, and it is the half that answers "show me what came from the server".

A Song Server tab came first, `yarg` `fbb36f3a`. The feature now has its own settings tab rather than three rows wedged under a header on Song Manager: URL, reachability, what this machine holds, the startup toggle, and Sync / Cancel. During a sync the mirror line becomes live progress.

A tab rather than a Main Menu screen, decided on cost. Menus are `MenuObject` children inside `Scenes/MenuScene.unity` keyed by a `MenuManager.Menu` enum, so a Main Menu entry means hand-building a GameObject subtree in a scene file — the one kind of change in this work that cannot be checked by measuring, only by looking at it. A tab follows the `SongManagerTab` precedent and inherits navigation, layout and controller input. **It needs no new prefab**: every row is a type that already existed, including the live text row added with the status line and a `ButtonRowMetadata` that already took `params string[]`.

Two new kinds of check came out of it, both worth reusing:

- **Tab icons are validated against the real sprite atlas.** They are Addressables lookups by string (`TabIcons[Import]`), so a name that is not in the atlas compiles, runs, and produces a tab with no icon that nobody notices until a screenshot.
- **The mirror row is counted against a folder that really holds songs**, not only the empty case: "Mirrored: 23 songs, 233 KB on disk" against 23 files, and 238,215 bytes is 233 KB.

End-to-end against vault2 after the `Sync` signature changes: `server=23 had=0 downloaded=23 failed=0 bytes=238215` — byte-identical to the baseline from when the mirror first shipped.

Server status in the menu came first, `yarg` `c025a98a` **.** Settings -> Song Manager shows a live line under the URL: connected and how many songs, or not reachable and why, plus the last sync's outcome. Before this, the only way to learn whether a URL worked was to press Sync and read an error dialog — a typo, a sleeping NAS and a healthy server were indistinguishable until you committed to a sync.

It reports **the server's own health too**, because `/api/v1/library` already names every directory the scan could not read and a library that quietly indexes 9,000 of 10,000 songs looks exactly like a library that has 9,000 songs. It shows `distinct_charts`, not `songs`: the server counts packages, two packages sharing a chart are one song to YARG, and the package count would promise more songs than a sync can deliver. Verified against vault2 — the row reads "connected, 23 songs" and the server independently reports `distinct_charts: 23`.

One probe in this change went green while testing nothing, and that is the reusable part. `SettingsManager.Settings` is null outside a running game; the status check dereferenced it before setting any state, and the `NullReferenceException` was swallowed by a fire-and-forget call. The state never changed, so both assertions held vacuously — the log carried four NREs while the probe reported PASS. The fix was not only to null-guard the read (a real defect: the row is reachable from a menu drawn before settings load, and would have stayed blank forever while looking like it worked) but to make the probe call the same path directly, with nothing able to swallow an error. **A probe that cannot fail is not evidence**, and the tell was in the log the whole time — lesson 5, "a green must come from the thing being tested", found again in a new disguise.

Auto-sync on startup came first, `yarg` `6a6888de` **.** Pointing the game at a server is now something you do once: `Sync On Startup` (default on) mirrors before the startup scan, so a machine that plays does not have to be administered. Default-on costs nothing until a URL is set -

the path returns before opening a socket.

Reading YARG.Core is what made it work, and the fact is worth keeping: a startup **quick scan cannot see new files.** `CacheHandler.QuickScan` only deserialises `songcache.bin` - its own summary is "*performing very few validation checks ... for the sole purpose of speeding through to gameplay*" - and never walks the filesystem; it falls through to a full scan only when it parses **zero** entries. Syncing before the ordinary startup scan would therefore have downloaded songs that stayed invisible until the player manually refreshed, and the feature would have looked broken while working perfectly. The scan mode is now decided by what the sync did: full when something arrived, quick otherwise.

Startup is the one place this must be neither loud nor slow. A song server is somebody's Pi or NAS and will be off, asleep or behind a dropped link a good fraction of the time, which is the ordinary case rather than the exceptional one. Every startup failure is logged and swallowed - no dialog in front of a loading screen - and reachability gets its own budget separate from download time: `Sync` takes `listTimeoutSeconds`, and startup passes 5 instead of 30. **Measured at 5.1 s** by `Assets/Editor/StartupReachabilityProbe.cs` against `192.0.2.1` (TEST-NET-1, RFC 5737, guaranteed unroutable). Deliberately not a closed port on localhost: a refused connection returns instantly and would pass the test while proving nothing, where an unroutable address hangs, which is what an unplugged Pi actually does.

The settings row came first, on 2026-09-07. Increment 1 shipped with `SongServerUrl` as a hidden field edited by hand in `settings.json`, because the project had exactly one `AbstractSetting<string>` — the IPv4 one — and its visual parsed IPv4 addresses itself. It is now a real row on the Song Manager tab:

- `TextSetting` is the shared base; the setting owns what is valid, the visual owns the text field and knows nothing about either.
- `IPv4SettingVisual` became `TextSettingVisual`, **renamed rather than replaced so its `.cs.meta` GUID survives** — that GUID is the prefab's reference to it and to its four serialized fields, and re-creating the file would have silently emptied all of them.
- Every text setting shares the one prefab. No Addressables change, no cloned asset.

The reuse plan was wrong in one place, and only reading the asset found it. The shared prefab has `m_CharacterValidation: 6` (Regex) with `m_RegexValue: '[\d.]'` baked in, so a URL is not merely rejected in that field — it is **untypeable**, the letters silently swallowed. TMP exposes no setter for `m_RegexValue`, so the filter is now installed through the public `onValidateInput` delegate, which takes precedence over the serialized one (`TMP_InputField.cs:631`, `onValidateInput ?? Validate`).

Verified by `Assets/Editor/SettingsRowProbe.cs` in batchmode, 9 checks, zero `error CS`: the prefab resolves `TextSettingVisual`, all four serialized references survived the rename, `onEndEdit` still reaches `OnTextFieldChange`, the localization keys are in `Settings.Setting` and not a neighbouring section, **a URL is typeable in the row**, and the IPv4 row still rejects letters and accepts digits.

One incidental measurement, upstream's behaviour and not ours: `IPAddress.TryParse` reads a leading-zero octet as **octal**, so typing `010.1.1.1` into the RB3E or sACN row silently stores `8.1.1.1`. The probe pins it so a future change to it is visible. A comment in `IPv4Setting` claimed the opposite until this was measured.

Exit criterion: the fork can browse and play from a server without a sync step, and a discussion thread exists upstream.

Half met, and the built half is finished rather than merely working. The game mirrors from a server with no separate tool, syncs on startup by default so the step is invisible rather than absent, and the whole feature is administered from its own settings tab — URL, reachability, what this machine holds, progress and cancel — with `server:yes` filtering the library to what the mirror brought in. The integrity guarantee is tested against a server that lies rather than asserted in a comment.

The discussion thread is still waiting on Jay to post, and what it asks has changed: a working implementation plus two `git am`-ready bug fixes, rather than a proposal.

Phase 4 — Modular features and the server config menu

The point at which the server stops being one feature and becomes a platform.

- ~~Feature registry: each capability is opt-in and independently enable-able.~~ **Built 2026-09-08.** `config.Resolved` now carries, alongside the settings, **where each one came from** — `default`, `file` or `flag` — and `Features()` turns that into one ordered list of capabilities. It feeds two consumers so they cannot disagree: the startup log, which now reports **every** feature including the ones that are off, and `GET /api/v1/features`.

Two things this fixes that were not on anyone's list. **The server never named the config file it read**, so an operator who edited a file the server never opened — wrong path, wrong working directory, a bind mount that landed elsewhere — got a server behaving exactly as if the file did not exist and saying nothing about it. And **a feature that was OFF logged nothing at all**, so "off because you turned it off" and "off because nothing you wrote was ever read" were the same silence. Both are the shape this project keeps paying for: *detecting something and reporting nothing is indistinguishable from not detecting it.*

Provenance is recorded by the parser rather than derived by diffing the config before and after, because a file setting a key to the value it already had is **still the file speaking** — and that is precisely the case an operator asks about. A test pins it.

The registry is **descriptive, not the gate**: the route is still registered from the resolved config, and a test asserts the registry and the routes agree in both directions, because a page reading "check_uploads: enabled" and then getting a 404 turns a configuration mistake into an apparent server bug. Red-proofed: breaking `Features()` fails tests in all three packages.

Paths and the listen address are deliberately **absent** from the registry. They are settings, not capabilities, and the endpoint is unauthenticated — which capabilities exist is already discoverable, a server's filesystem layout is not. Asserted, because "just return the config" is the obvious future shortcut. - ~~Config UI in the server app, so a user turns on only what they want.~~ **Built 2026-09-08.** `PUT /api/v1/features/{name}` changes a capability with no restart, and the browse page carries a settings panel with a switch per feature. That closes the last clause of goal 1.

`config_writes = off | local | lan`, default `off`. Not a bool, and that is the decision worth keeping: "yes" would have had to mean `lan`, and on a home network that is every device including the ones nobody is thinking about. This server has no authentication, so `lan` means exactly what it says and belongs to an operator who chose it on purpose. `local` — a settings page that works at the machine and not from a phone — is the useful middle.

Three properties, each asserted rather than assumed:

- **A disabled feature is still ABSENT, not forbidden.** That used to be true by construction: the route was never registered. A runtime toggle cannot work that way, because a route that does not exist cannot be switched on, so the routes are always registered now and the handlers answer with `http.NotFound`. The observable contract is unchanged and only the mechanism moved. The test compares a disabled feature's WHOLE response — status, body, content-type — against a path that genuinely does not exist. A status-only check would have passed against a JSON error body, which would still have told the caller the feature was there.
- **The write surface cannot widen itself.** `config_writes` is not writable. One call turning `local` into `lan` would end the "only from this machine" promise, made by whoever was already inside it.
- **Locality comes from `RemoteAddr`, never `X-Forwarded-For`.** A header the caller supplies would make `local` a suggestion: anybody on the LAN types one line and is treated as sitting at the machine. Red-proofed — trusting the header fails the test.

A change is written back to the config file, so it survives a restart — added in a second increment, deliberately separate because editing an operator's commented file in place carries its own risk. `config.SetKey` changes one line and leaves every other byte alone.

The rule that took thinking: **last one wins in this format**, because `Apply` walks the file in order, so the LAST uncommented line for the key is the one rewritten. Rewriting the first would leave a later line silently overriding it, and the operator's file would say one thing while the server did another. Red-proofing "first match" fails exactly that test and nothing else, which is what makes the test worth having. With no uncommented line the setting is appended with a dated comment, leaving the commented template — which is documentation — untouched. The value is parsed back before anything is written, because persisting a line that stops the server starting next time is the worst outcome a convenience feature can have. Atomic via a temp file in the same directory; permissions and line endings preserved.

A failed save is not a failed toggle. The runtime change stands either way and the response reports the two halves separately. The server never *creates* a config file: settings written into whatever directory it happens to be running from would land somewhere nobody would look.

A test caught a bug that reading the code twice had not — the rewrite emitted the captured pre-`=` whitespace *and* a space of its own, giving `check_uploads = yes`. The alignment there is the operator's, not ours.

Verified through a restart, which is the only thing that proves persistence: clicking the page's own switch wrote `check_uploads = yes` under a dated comment, the template's `# check_uploads = no` survived with all 73 comment lines intact, and the restarted server reported `feature=check_uploads enabled source=file`. That last word is the confirmation — the registry's provenance says the value came from the file, because it now does.

The page **asks whether it may write** rather than assuming, the same rule as the drop zone, and re-reads the server after a change rather than trusting what it just sent. Verified in a real browser both ways: with `local`, clicking the page's own switch flipped the server and the drop zone appeared with no reload; at the default, the panel showed state, offered **zero** switches, and a PUT from that page answered 404. - Candidate modules: multi-user libraries and permissions, playlists/setlists shared across clients, scores and leaderboards, library health reporting.

The web UI: managing the library without launching YARG

This is goal 2 from the top of the document, and it is the half of the server most easily mistaken for a nice-to-have. A library that can only be inspected through the game is a library you can only inspect on a machine with the game installed, sitting in front of the TV, with the game running. Everything else — checking whether last night's ingest worked, finding out why four songs are missing, seeing what a charter actually named a track — means launching a rhythm game to read a list.

Shipped 2026-09-07. `GET /` serves a phone-friendly page: search, the twelve sort attributes read from `/api/v1/library` so the page cannot drift from the server, and a per-song panel with metadata, parts, the scanner's issues and a download link. Embedded in the binary — no CDN, no build step — because a Pi at a party has no internet to fetch a stylesheet from. On by default (`browse_ui`); it exposes nothing the API did not already.

Registered as `GET /{$}`, and that detail is load-bearing: a bare `GET /` in Go's ServeMux is a catch-all that would answer every unmatched path with the page and a 200, turning every documented 404 into HTML a sync client would try to parse as a `.sng`. Red-proofed.

It rests on `/api/v1/songs`, which already does free text across name, artist, album, genre, subgenre, charter, source and playlist, with twelve sort attributes, ordering and paging, and answers a 10,000-song catalog in ~140 ms.

Where this goes next, all of it server-side and none of it needing the game:

- ~~**Library health as a view, not a log line.**~~ **Done.** The page fetched `/api/v1/library` for its sort attributes and threw `problems` away, so a library missing a thousand songs looked exactly like a library that had a thousand fewer. It now shows "N items could not be read", collapsed by default with each path and reason inside — a healthy library must not be made to look alarming, and the count is the part that matters at a glance. Paths and errors come from the filesystem, so both are escaped. The page is now `browse.html`, not `party.html`: the name was left over from framing this project does not own.
- **Ingest from the browser** — drop an archive in, watch it scan, see the verdict. **Half of this shipped 2026-09-08: `POST /api/v1/check` answers the verdict and keeps nothing.** It exists because the question people actually arrive with is *"why won't this song work?"*, and until now the only way to ask it was to copy the file onto the server, rescan, and read the log — a shell session, on the machine, for something the scanner answers in milliseconds.

Off by default, unlike `browse_ui`, and the difference is the decision worth keeping: the browse page is a new *view* of surface the API already served to anyone who could reach the port, while this accepts a large upload and spends CPU and temp disk on whoever asks. An existing deployment must not acquire that by being upgraded. When it is off the route is not registered at all, so the answer is 404 — a 403 would tell the caller the feature is there.

The verdict comes from `scan.ScanFile`, extracted from `WalkLibrary` in the same change so that the dispatch deciding *what a file is* has exactly one implementation. Two would agree for a while and then quietly stop, and the disagreement would read as "the checker said it was fine and the library dropped it" — the failure this project has already paid for in the packcache path and in both sync clients. A test uploads a real `.sng` and compares its chart hash against a library built from the same bytes.

The traversal test had to be written twice, and the first version was green for the wrong reason. It listed the staging directory's parent before and after — and passed against an implementation that really did join the uploaded name onto the staging path, because that implementation created the file outside and then the handler's own cleanup deleted it again before the test looked. Identical listings, green test, a server writing exactly where it was told. The damage `os.Create` actually does is TRUNCATION of a file already there, so the test now plants a sentinel where a traversal would land and asserts it still says what it said. Both plausible buggy implementations fail it now, and the failure message shows the operator's file being deleted outright. Same shape as the first hostile-server test, which proved nothing because the server 404'd the names it was meant to serve — **the second time in this project that a security test passed against the vulnerable code.**

The drop zone followed the same day. An endpoint with no UI still means a `curl` command, which is not "managing the library without launching YARG" for anyone who is not already at a shell. `GET /` now offers a drop zone, and `/api/v1/library` gained `check_uploads` so the page can ask whether the server will answer rather than assume it — capability from the server, the same rule that already governs `sort_attributes`. Files go one at a time: a dropped folder can be hundreds, and the target is a Pi.

Driven in a real browser against a real server, because reasoning about escaping is exactly how an audit goes wrong. Files were pushed through the page's own input handler, not through `fetch` written for the occasion: a real `.sng` pulled out of that library came back accepted with a matching hash and "Already in this library"; `garbage.sng`, `holiday-photos.rar` and `Some Song_rb3con` were each refused with their own reason; a file named `<img src=x onerror="document.title='PWNED'">.sng` injected **0 elements** and left the title alone; and the staging directory was **empty** afterwards, so "keeps nothing" is now measured on a live server rather than in a unit test.

Upstream contact is deferred to the bottom of the queue (Jay, 2026-09-08): nothing goes to YARG's team until there is something to show, and that means a public beta. See `UPSTREAM.md`.

And upstream is a collaborator we would welcome, not a gate we are waiting at — Jay, the same day. ADR-004 increments 2 and 3 are ours to decide, and the first thing that produced was a measurement rather than a decision: **increment 2 does not need `YARG.Core` after all, and should be SKIPPED anyway.** An entry survives a quick scan with its file deleted, so the Unity layer could fetch on play with no upstream change — but every player-facing refresh in the game is a FULL rescan (four call sites, all `quick: false`), which drops every unfetched entry, and the game's own "*Chart requires a rescan!*" path sends players straight at the button that does it. Increment 2 is increment 1 plus a trick the game itself undoes. The `YARG.Core` conversation belongs at increment 3, where entries would come from the scanner rather than from the cache.

Storing an upload is ADR-005 increment 2 and is deliberately not built: `--songs` is mounted `ro` on the live deployment, an unauthenticated write endpoint is a different risk from an unauthenticated read one, and the in-memory index (ADR-002) would need a rescan path. None of that is hard; it is just undecided, and increment 1 is useful without any of it. - **Metadata and playlists**, once multi-user libraries exist (Phase 4 proper).

What it deliberately does not do: control a running game. No queue, no remote select, and no UI hinting at either. That capability lives inside YARG and upstream has both an open request (#860) and a stalled draft PR (#984) in that space — see `UPSTREAM.md`, where it belongs as collaboration context. If we ever want it, it is a conversation with their team, not a feature we bolt on from outside.

Shipping the client — measured 2026-09-08

Nobody had ever built this fork. Every measurement until now was against the official v0.15.0 release or in editor batchmode, and "it compiles headless" is not "it ships". A public beta means somebody who is not Jay running the client, so the first question is whether a build exists at all.

It does. **526 MB in 34.1 minutes**, Unity 6000.3.5f2, `StandaloneWindows64`, Mono backend, and the fork's own code verified present in `Assembly-CSharp.dll` rather than assumed — `SongServerSync`, `MirroredSongs`, `SongServerTab`, `WithServerBadge`.

Three things came out of it that matter beyond "it worked", all in `docs/BUILD.md`:

- **Upstream has no Windows build workflow at all**, and its one mac workflow pins Unity `2021.3.21f1` against the project's real `6000.3.5f2`. There was no recipe to copy.
- **Unity returned `Success` WITH three errors.** All `RenderTexture.Create failed`, from `-nographics`. Almost certainly harmless, and "almost certainly" is the honest strength of that claim, because nobody has launched the build. The reporting now says `PASS WITH 3 ERROR(S)` rather than calling it clean — `BuildPipeline.BuildPlayer` returns `Succeeded` alongside a non-zero error count, so a script that only read the result would have lied.
- **The binary identified itself as `YARC` / `YARG`**, upstream's own values — so a distributed executable said nothing about being a fork, and upstream would have fielded our bug reports. **Settled 2026-09-08** (fork commit `bc5847ca`): `FatalException` / `YARG-FE`. The label was the smaller half; Unity derives `Application.persistentDataPath` from those strings, so the fork had been writing `settings.json` and `songcache.bin` into an official install's folder. `Assets/Editor/IdentityProbe.cs` asserts the *moved folder*, not the edited strings, and also asserts that `PathHelper.LauncherPath` did NOT move — that one points at the external YARC Launcher and must keep doing so. See `docs/BUILD.md`.

Shipping the server — measured 2026-09-08

There was no Windows app, and there had never been one. Measured rather than assumed: no tray, service, GUI or installer code anywhere in the repo, and no GUI dependency in `go.mod`. What a Windows user could get was a console `.exe` inside a CI artifact that **expired after a week**, run from a terminal with flags. That is a build output, not something to hand anyone.

`cmd/mkrelease` **now produces the download.** One archive per platform — `.zip` for Windows, `.tar.gz` elsewhere — each carrying **both binaries**, a per-platform README, the config template, the licence and a launcher, inside a folder named after itself. Plus `SHA256SUMS`. Artifacts from `main` keep 30 days; a tagged build keeps its archives permanently.

Decisions worth keeping:

- **Go, not `zip` and `tar`.** The runner is a shell executor and neither tool is guaranteed to be on the host. "The packaging step failed because the runner lacks a binary" is worth designing out rather than discovering.
- **The shipped `yarg-song-server.conf` IS `config.Example`**, not a copy kept beside it. A second copy of a settings file agrees with the real one for a while and then quietly stops, and the person it misleads is the one who trusted the file in the download.
- **The packaging is deterministic** — every entry is stamped with a fixed date rather than the clock, so the same binaries always produce the same archive. Otherwise `SHA256SUMS` says nothing: every rebuild would differ, and a real change would be indistinguishable from one.

That is a claim about the packaging, not the whole build, and chasing the difference found two real defects — neither of them the one first blamed.

The local and CI archives for `4ddd322` did not match. First hypothesis: **CI pinned Go 1.27.1 and ENG-1 had 1.27.0**, a drift the CI comment ("keep this in step with the toolchain the workstations use") exists to prevent. Real, and fixed — ENG-1 is now on 1.27.1, installed the same verified way (official zip, SHA-256 checked before extraction, old install kept at `Programs\go-1.27.0` so rollback is a rename).

It was not the cause. All six archives still differed on the matched toolchain. The actual cause is that nothing used `-trimpath`, so every binary embedded the absolute paths of the machine that built it: a plain `yarg-sync` carried **906 occurrences of `C:/Users/<name>`**, 770 of the local `GOROOT` and 18 of the project directory. With `-trimpath`: zero, and 37,376 fewer bytes.

That is two problems, not one. Reproducibility was the one being chased; the other is that **artifacts this project mirrors publicly were shipping the builder's username and directory layout**. `-trimpath` is now on every build path — Makefile, the release job and the Dockerfile.

A note on how the first measurement lied: searching the binary for `C:\dev\YARG` returned **0** and nearly closed the question. Go normalises embedded paths to **forward** slashes; the same search for `C:/dev/YARG` returned 18. The size delta between the two builds was the thing that said "something is in there", and it was the only honest signal until the separator was fixed.

It took four causes, and each of the first three looked like the answer. The discipline that mattered was refusing to stop at "the archives differ, probably because X" and instead diffing the artifacts until they were identical:

#	Hypothesis	Verdict
1	Go version drift (CI 1.27.1, ENG-1 1.27.0)	Real, fixed — **not the cause**, all six still differed
2	No `-trimpath`, so absolute paths are embedded	Real, fixed — **not the cause either**
3	`vcs.modified=true` from a phantom-dirty tree	Real, fixed — made `yarg-sync.exe` identical, **but not `yarg-song-server.exe`**
4	CRLF reaching embedded content	**The rest of it** — see below

The fourth is the one worth the whole exercise, because it was never about checksums. `internal/httpapi/web/browse.html` is `go:embed`ed, and a per-extension `.gitattributes` rule had missed `.html` — so **the browse page the server serves had different bytes depending on which operating system built the binary**: 15,653 from Windows, 15,302 from Linux, same commit. `LICENSE` differed by 164 bytes for the same reason. Fixed with `* text=auto eol=lf`, with the testdata fixtures still excluded and their hashes verified unmoved.

Result, measured on `b90cc3e`: a Windows workstation and the Linux CI runner produced byte-identical archives for all six platforms. 6 match, 0 differ. Reproducibility is now a property this project can check rather than a sentence in a README. - **The launcher does not**

open a browser for you. Launching a URL before the server is listening gives connection-refused, and the person then believes the download is broken when it is merely one second early. It prints the address instead. - **No GitLab Release object.** That needs `release-cli` on the runner host, which is the gitlab-ce session's box rather than this project's to change.

The determinism test had to be written twice, and the first one was green against broken code. It built twice in one process and compared bytes — which cannot catch `var epoch = time.Now()`, because a package-level var is evaluated once and both runs share it. It now asserts the stored timestamps against a date written in the test itself. Third time in this project that a test passed against the very defect it existed to catch.

Verified by doing it, not only in tests: twelve binaries built, packaged, the Windows zip extracted into a clean folder and `start-server.cmd` run as a double-click would. Server up, `GET /` 200 at 15,548 bytes, `/api/v1/features` and `/api/v1/library` both 200, version stamped, and the startup log naming the config file it read. Then the same thing again in CI.

Still not an app, and that is the honest description: it is a proper download. A tray icon and a service install are the next question. (The configuration menu that goal 1 asks for is now built — see phase 4 above.)

The whole path, run as a new user would, 2026-09-08

Every previous measurement had used a binary built on the machine doing the measuring. This one started from the published artifact and touched nothing else.

Step	Result
Download the CI artifact	7,773,966 bytes
Check it against the published `SHA256SUMS`	**match**
Unzip cold	six files, nothing scattered
Put 23 songs in `songs/`, double-click `start-server.cmd`	server up, `songs=23 problems=0`
`yarg-sync.exe -dry-run` from a **separate folder**	"would download 23", **0 files written**
`yarg-sync.exe` for real	23 songs, **238,215 bytes**, 0 failed
Run it again	**0 downloaded, 23 already present**
What landed	23 files, every one `<40 hex>.sng`, first bytes `SNGPKG`, **0 leftover** `.part`
Serve the SYNCED folder with a fresh server	`songs=23 distinct_charts=23 problems=0`

Step	Result
Compare chart hashes between the two servers	**identical sets, 23 vs 23**

Two things worth pulling out. **238,215 bytes is the same total four earlier sync clients pulled** and the same the fork's batchmode run reported — reproduced here by a server rebuilt from scratch weeks later, which is what deterministic packing was for. And the last two rows are the round trip: a loose folder packed to `.sng`, served over HTTP, written by a client, and rescanned by a different server still has **the same chart identity**. Identity survives the whole path, measured rather than assumed.

The launcher, the README's instructions, the config template and the checksum file were all exercised as written. Nothing in the archive needed correcting.

Phase 5 — LLM chart generation

The long-term goal, and the phase most likely to move. It depends on every phase above working.

- **Start from the tools that already exist.** Help:Charting names two, both free, that do the first half of this: **Ultimate Vocal Remover (UVR)** separates a mix into bass, drums, vocals and other stems, and Spotify's **Basic Pitch** generates pitched MIDI from a vocal stem — a baseline vocals chart to work from. The novel part of this phase is the instrument charting and the difficulty reduction, not stem separation, and building either from scratch would be waste.
- Upload any song; auto-generate instrument and vocal parts across all difficulty levels.
- Produce a complete, working, **editable** package — generated charts are a starting point, not a final answer.
- **Distribution constraint, non-negotiable:** the chart/vocal tracks must be packageable and distributable *separately from the audio*, so charts can be shared without the music.
- Note what YARN's guidelines establish about this: **visual synchronisation is itself a derivative work**, which is why they refuse no-derivatives licences. A generated chart is a derivative of the song it was generated from, not a neutral companion to it — so separating chart from audio reduces the problem, and does not by itself dissolve it.
- Runs against the existing GPU arbitration on the home estate rather than a new stack.

Client track (independent of the server line)

Can be picked up at any time; does not block the server.

- **Controller compatibility** — official Rock Band and Guitar Hero instruments first. Upstream handles this through PlasticBand-Unity, so fixes may belong there rather than in YARG.
- **Graphics** — general rendering and visual improvements.
- Both are ordinary upstream contributions: fork, branch off `dev`, PR to `dev`.

Blockers

None.

Toolchain and credentials, as measured

The Coffeehedake GitHub PAT was rotated on 2026-09-05 and verified (`login=Coffeehedake`, scopes `repo`, `workflow`, `project`, `write:packages`, `delete:packages`, `audit_log`).

Go 1.27.0 was installed on ENG-1 on 2026-09-05, so the toolchain claim is now measured on the machine the work actually happens on: `gofmt` clean, `go vet` exit 0, `go test ./...` green, and all six release targets building — linux/amd64, linux/arm64, linux/armv7, darwin/amd64, darwin/arm64, windows/amd64.

It is a **per-user** install at `%LOCALAPPDATA%\Programs\go`, from the official zip with its SHA-256 verified against `go.dev/dl/?mode=json` before extraction, with `go\bin` and `%USERPROFILE%\go\bin` added to the user `Path`. Per-user rather than machine-wide on purpose: the MSI needs elevation, and an unattended `msiexec /qn` from a non-elevated session fails *silently* — the same failure mode that produced the Bambu Studio update loop. Nothing about this install needs elevation, and `GOPATH` / `GOCACHE` land in the user profile rather than in the projects root — which mattered when that root was a Syncthing folder, and now simply keeps a multi-gigabyte build cache out of a directory full of repos. Promote it to a machine-wide install later if you want; nothing depends on where it lives.

mingw-w64 followed, for the race detector. `go test -race` needs cgo, and cgo needs a C compiler; without one ENG-1 reported `CGO_ENABLED=0` and `-race` failed outright with "requires cgo" rather than passing quietly. WinLibs GCC **16.2.0** (UCRT, POSIX threads, SEH) is now installed the same way — per-user at `%LOCALAPPDATA%\Programs\mingw64`, SHA-256 verified against the publisher's own `.sha256` before extraction, `bin` on the user `Path`. `go env CGO_ENABLED` now reports 1 and the suite is race-clean in about 16 seconds on the first run.

The extraction ran through a **scheduled task** rather than `Start-Process`, because a long child process started from a Cowork session dies with the process tree when a bridge call times out — that is what silently cancelled an earlier `winget` attempt mid-download. A scheduled task is detached from that tree and survives.

And the detector was proved able to go **red** before its green was believed: a throwaway module with eight goroutines incrementing a shared int returned `WARNING: DATA RACE` and exit 1.

Sequencing note

Phases 1 and 2 are the whole of the "#1 priority". Nothing in phases 3–5 should start before a stock YARG client is reading songs from a self-hosted server, because every later decision depends on what that turns out to actually require.
